ProbOS — Month 3, Week 10
GPU Kernel Path

Nisong Monyimba

July 4, 2026

Week 10 goal

Build and honestly benchmark a GPU-accelerated path for `MonteCarloEngine`'s forward-simulation loop, per Month 3's own stated priority order and an explicit decision to finish kernel/speed/core work before further feature work (PDSL v0.2, etc.).

Hardware confirmed before writing this plan: NVIDIA GeForce RTX 3050 (4GB VRAM), CUDA 13.3 drivers already working correctly through WSL2 via `nvidia-smi` – no additional driver setup needed. Genuinely sufficient for ProbOS's typical particle counts (N = 5,000–100,000).

Critical constraint, stated up front: GitHub Actions CI runners do NOT have GPU access. Any GPU-specific code must be written to gracefully skip (via `pytest.mark.skipif`) when no GPU is present, and CI will only validate that the GPU-path code exists and does not break anything else – genuine GPU correctness and performance validation can only happen on this local machine, not CI. This must not be papered over or silently assumed to work.

Day-by-day plan

Day 1 – Environment setup and feasibility check

- Install CuPy matched to the confirmed CUDA 13.x toolkit version (CuPy chosen over raw Numba CUDA kernels: it is a near-drop-in NumPy replacement, meaning `BatteryModel2Cell.forward_batch()`'s existing vectorised NumPy code needs minimal rework, consistent with keeping Week 4's original model unchanged wherever possible)
- Verify basic GPU array operations work correctly from within the existing `.venv` (no engine changes yet) – a genuine feasibility smoke test, not assumed
- Confirm `cupy.cuda.is_available()` and basic array-transfer round-trip correctness (CPU array -¿ GPU -¿ CPU, values unchanged)

Day 2 – GPU-compatible forward_batch

- Determine whether `BatteryModel2Cell.forward_batch()`'s existing NumPy operations are already CuPy-compatible as written, or need a thin dispatch layer (array-module parameter, following the common `xp = cupy if on_gpu else numpy` pattern)
- Initial correctness check: same inputs, CPU (NumPy) vs GPU (CuPy) outputs, at an appropriate tolerance (GPU computations commonly default to float32 unless explicitly configured for float64 – this must be checked explicitly, not assumed to match CPU float64 precision)

Day 3 – GPUMonteCarloEngine

- Build a new class following `MonteCarloEngine`'s exact existing architecture (same constructor signature, same `MCResult` return type extended if needed), but running the simulation loop with CuPy arrays on the GPU
- Existing `Model`/`Distribution` ABCs reused unchanged – domain-agnostic kernel design already established should extend to the GPU path without modification

Day 4 – Correctness validation

- Full `GPUMonteCarloEngine` run vs `MonteCarloEngine` (CPU) run, same seed, same N, checked at an honestly-determined tolerance (not blindly `rtol=1e-10` if float32 GPU precision cannot support it – this must be determined empirically, not assumed)

Day 5 – Honest benchmarking

- Measure real wall-clock speedup (or lack thereof) vs both the Python engine and the existing C++/OpenMP engine, across a range of N (small N may show GPU overhead DOMINATING actual compute – kernel launch and host-device transfer overhead can make small problems slower on GPU, not faster; this must be measured, not assumed away)
- Per the project's established "measure, don't assume" discipline (Month 1 Week 4's DP profiling found one genuine non-win alongside two large wins) – report the real numbers honestly, including if GPU turns out not to help at ProbOS's typical N for this specific workload

Day 6 – Test coverage, pre_week_audit.sh, CI

- Tests must skip gracefully in CI (no GPU) via `pytest.mark.skipif(not cupy_available, ...)`, while still running for real locally
- Confirm `pre_week_audit.sh` and CI both pass with the new code present but GPU tests skipped – CI must not silently report success without actually running anything meaningful for this feature; the skip must be visible in the test output, not hidden

Day 7 – Documentation + retrospective

- Update `docs/sbir_phase1_onepager.md`: move the GPU kernel item from the SBIR Ask section to demonstrated results IF genuinely complete and shown to help; if the honest benchmark result is negative or mixed, document that accurately instead of overselling
- Retrospective appended to this document once complete

Exit criteria

A working `GPUMonteCarloEngine` exists, is proven numerically correct against the CPU engine at an honestly-determined tolerance, and its real performance characteristics (whether positive, negative, or mixed across different N) are measured and reported truthfully – not assumed, and not silently downplayed if unfavourable.